

 Aula 3

Aula 3!



Curso de Programação em Java



Curso

Programação em Java



Módulo

Módulo 1 – Fundamentos



Arrays



Strings



Aula 3

Arrays & Strings



Σ

Tt

Recapitulando a Aula 2



Na aula passada aprendemos:

📁 Estruturas Condicionais

if else else if switch

🔄 Laços de Repetição

for while do-while

🔴 Controles

break
continue

📁 Operadores

&& || !



Hoje vamos aprender:



Arrays

Listas de dados do mesmo tipo



Strings

Manipulação avançada de textos



O Problema – Guardando Muitos Valores

Imagine que você quer guardar as notas de 5 alunos

✗ Jeito ruim (sem array)

```
double nota1 = 7.5;  
double nota2 = 8.0;  
double nota3 = 6.5;  
double nota4 = 9.0;  
double nota5 = 5.5;
```



E se fossem 100 alunos?

1000 alunos? 🤖



✓ Jeito inteligente (com array)

```
double[] notas= {  
    7.5, 8.0, 6.5, 9.0, 5.5  
};
```



Uma variável para todos

Array guarda múltiplos valores



Fácil acesso por índice

notas[0], notas[1], notas[2]...

O que é um Array?



Definição

Array = uma variável que guarda **múltiplos valores do mesmo tipo**, em posições numeradas.

Regras Importantes



Índices começam em 0

Não em 1!



Mesmo tipo

Todos os elementos são do mesmo tipo



Tamanho fixo

Definido na criação, não muda

Visualização do Array

 notas

notas: _____

Índice 0	Índice 1	Índice 2	Índice 3	Índice 4
7.5	8.0	6.5	9.0	5.5
[0]	[1]	[2]	[3]	[4]



Para acessar: `notas[0]` → 7.5

Use o índice entre colchetes

Declarando e Criando Arrays



Forma 1

Inicializar junto

```
int[] numeros = {  
    10, 20, 30, 40, 50  
};  
  
String[] nomes = {  
    "Ana", "Bruno", "Carlos"  
};
```

✓ Mais rápido e direto



Forma 2

Definir tamanho depois

```
int[] notas =  
    new int[5];  
  
// cria 5 posições (todas = 0)  
  
notas[0] = 10;  
notas[1] = 8;  
notas[2] = 7;
```

✓ Flexível para dados dinâmicos



Acessando

Usar índice

```
System.out.println(  
    numeros[0]);
```

→ // imprime 10

```
System.out.println(  
    nomes[1]);
```

→ // imprime Bruno

✓ Índice entre colchetes

Percorrendo Arrays com For

<> Exemplo de Código

```
1 int[] numeros= {10,20,30,40,50};  
2  
3 for(int i=0;i<numeros.length;i++) {  
4     System.out.println(  
5         "Posição "+i+": "+numeros[i]  
6     );  
7 }
```



Dica Importante

`numeros.length` → retorna o tamanho (5)

Resultado

Posição 0: 10

Posição 1: 20

Posição 2: 30

Posição 3: 40

Posição 4: 50

Array na Memória

[0]

10

[1]

20

[2]

30

[3]

40

[4]

50

For-Each – Loop Simplificado ✨

<> Exemplo de Código

```
1 String[]frutas= {"Maçã","Banana","Laranja","Uva"};
2
3 for(Stringfruta:frutas) {
4     System.out.println(
5         "Fruta: "+fruta
6     );
7 }
```



Como ler

"Para cada **fruta** em **frutas**, faça..."

📄 Resultado

Fruta: **Maçã**

Fruta: **Banana**

Fruta: **Laranja**

Fruta: **Uva**



Vantagens

- ✓ Mais legível
- ✓ Menos propenso a erros
- ✓ Ideal para apenas ler dados

Exemplo Prático – Média de Notas

<> Código Completo

```
1  import java.util.Scanner;
2
3  public class MediaNotas{
4      public static void main(String[] args) {
5          Scanner sc = new Scanner(System.in);
6          int[] notas = new int[5];
7          int soma = 0;
8          // Preenchendo o array
9          for(int i = 0; i < notas.length; i++) {
10             System.out.print("Digite a nota " + (i+1) + ": ");
11             notas[i] = sc.nextInt();
12             soma += notas[i];
13         }
14         double media = (double) soma / notas.length;
15         System.out.println("Média da turma: " + media);
16         sc.close();
17     }
18 }
```

+ Criando o Array

```
int[] notas = new int[5];
```

Reserva espaço para 5 inteiros

↻ Preenchendo e Somando

- Pede nota ao usuário
- Guarda no array
- Acumula na soma

Σ Calculando Média

```
double media = (double) soma /
notas.length;
```

 Cast (double) evita perda decimal

Maior e Menor Valor do Array

<> Código Completo

```
1  int[] numeros= {45,12,78,23,56,9,90};
2  int maior=numeros[0];
3  int menor=numeros[0];
4
5  for(int num:numeros) {
6      if(num>maior) {
7          maior=num;
8      }
9      if(num<menor) {
10         menor=num;
11     }
12 }
13
14 System.out.println("Maior: "+maior); // 90
15 System.out.println("Menor: "+menor); // 9
```

💡 Lógica Importante

- Inicializar com `numeros[0]`
- ⚠ Não usar 0 (array pode ter negativos)



MAIOR
90



MENOR
9

⚙ Como Funciona

- 1 Percorre cada número
- 2 Compara com maior/menor atual
- 3 Atualiza se necessário

Arrays Multidimensionais (Matrizes) ▮

<> Criando e Acessando Matriz

```

1  int[][]matriz= {
2      {1,2,3},
3      {4,5,6},
4      {7,8,9}
5  };

6
7  // Acessando: matriz[linha][coluna]
8  System.out.println(matriz[0][0]); // 1
9  System.out.println(matriz[1][2]); // 6
10 System.out.println(matriz[2][1]); // 8

11
12 // Percorrendo com loops aninhados
13 for(int i=0;i<3;i++) {
14     for(int j=0;j<3;j++) {
15         System.out.print(matriz[i][j]+" ");
16     }
17     System.out.println();
18 }
    
```

Visualização 3x3

[0][0] 1	[0][1] 2	[0][2] 3
[1][0] 4	[1][1] 5	[1][2] 6
[2][0] 7	[2][1] 8	[2][2] 9

💡 Conceito Chave

- `matriz[linha][coluna]`
- Como uma planilha Excel
- Útil para jogos (jogo da velha)

Strings – Muito Além do Texto! 📖

<> Exemplos de Métodos

```
1 String nome = "Maria Silva";  
2  
3 // Informações sobre a String  
4 System.out.println(nome.length()); // 11 (tamanho)  
5 System.out.println(nome.toUpperCase()); // MARIA SILVA  
6 System.out.println(nome.toLowerCase()); // maria silva  
7 System.out.println(nome.charAt(0)); // M (primeiro caractere)
```



Lembre-se

String começa com letra **maiúscula** porque é uma **CLASSE**!



length()

Retorna o número de caracteres



toUpperCase()

Converte para MAIÚSCULAS



toLowerCase()

Converte para minúsculas



charAt(int)

Retorna caractere na posição

Métodos Essenciais de String (Parte 1) 📄

<> Exemplos Práticos

```
1 String frase = " Olá, Mundo Java! ";
2
3 // Verificando conteúdo
4 System.out.println(frase.contains("Java")); // true
5 System.out.println(frase.startsWith("Olá")); // false
6 System.out.println(frase.endsWith("!")); // false
7
8 // Limpando espaços
9 String limpa = frase.trim();
10 System.out.println(limpa); // "Olá, Mundo Java!"
11 System.out.println(limpa.startsWith("Olá")); // true
12
13 // Posição de um trecho
14 System.out.println(limpa.indexOf("Mundo")); // 5
```



contains(String)

Verifica se contém o trecho

Retorna: true/false



startsWith(String)

Verifica se começa com

Retorna: true/false



endsWith(String)

Verifica se termina com

Retorna: true/false



trim()

Remove espaços nas extremidades

Importante para validação!

Métodos Essenciais de String (Parte 2)



<> Exemplos Práticos

```
1 String texto = "Programação em Java é incrível!";
2
3 // Recortando
4 System.out.println(texto.substring(0, 11)); // Programação
5 System.out.println(texto.substring(15)); // Java é incrível!
6
7 // Substituindo
8 String novo = texto.replace("incrível", "poderoso");
9 System.out.println(novo); // Programação em Java é poderoso!
10
11 // Dividindo
12 String csv = "Ana,Bruno,Carlos,Diana";
13 String[] nomes = csv.split(",");
14 for (String nome : nomes) {
15     System.out.println(nome);
16 }
```



substring(início, fim)

Extraí parte da String

Fim é exclusivo!



replace(antigo, novo)

Substitui trecho

Útil para sanitizar dados



split(separador)

Divide String em array

Essencial para CSV!

⚠️ Atenção Importante

Comparando Strings – Cuidado! ⚠️

<> Exemplos de Comparação

```
1 Stringa="Java";
2 Stringb="Java";
3 Stringc=newString("Java");

4
5 // Comparação ERRADA (compara referência de memória)
6 System.out.println(a==b);// pode ser true ou false!
7 System.out.println(a==c);// false ✗

8
9 // Comparação CORRETA (compara conteúdo)
10 System.out.println(a.equals(b));// true ✔
11 System.out.println(a.equals(c));// true ✔

12
13 // Ignorando maiúsculas/minúsculas
14 System.out.println(a.equalsIgnoreCase("java"));// true ✔
```



NUNCA use ==

Compara referência de memória

a == b ✗



Use .equals()

Compara conteúdo da String

a.equals(b) ✔



.equalsIgnoreCase()

Ignora maiúsculas/minúsculas

Perfeito para buscas e login!

Convertendo Tipos com String

<> Exemplos Práticos

```
1 // String → Número
2 String numStr="42";
3 int numero=Integer.parseInt(numStr);
4 double decimal=Double.parseDouble("3.14");
5 System.out.println(numero+10); // 52 (soma numérica)
6 System.out.println("42"+10); // 4210 (concatenação!)
7
8 // Número → String
9 int valor=100;
10 String valStr=String.valueOf(valor); // "100"
11
12 // Verificando se é número
13 try{
14     int x=Integer.parseInt("abc"); // lança exceção!
15 }catch(NumberFormatException) {
16     System.out.println("Não é um número válido!");
17 }
```



String → Número

Convertendo texto em número

```
Integer.parseInt("42")
Double.parseDouble("3.14")
```



Número → String

Convertendo número em texto

```
String.valueOf(100)
Integer.toString(100)
```



Cuidado com concatenação!

"42" + 10 = "4210" (não 52!)



Use `parseInt()` para soma numérica

StringBuilder – Construindo Textos Eficientemente 📄

Por que usar? Strings em Java são **imutáveis** – cada concatenação cria um novo objeto!

✗ Ineficiente

Cria novo objeto a cada concatenação

```
String resultado = "";
for (int i = 1; i ≤ 5; i++) {
    resultado += "Item " + i + "\n"; // novo objeto!
}
```

✓ Eficiente

Modifica o mesmo objeto

```
StringBuilder sb = new StringBuilder();
for (int i = 1; i ≤ 5; i++) {
    sb.append("Item ").append(i).append("\n");
}
System.out.println(sb.toString());
```

🔑 Principais Métodos

append()

insert()

delete()

reverse()

💡 Quando usar?

- ▶ Construir strings dentro de loops
- ▶ Muitas concatenações
- ▶ Manipular texto frequentemente

Exemplo Completo – Processando Nomes

Combinando **Arrays** + **Strings** em algo útil

<> Código Completo

```
import java.util.Scanner;

public class ProcessaNomes {
    public static void main(String[] args) {
        Scanner sc = new Scanner(System.in);
        String[] nomes = new String[4];

        // Coleta
        for (int i = 0; i < nomes.length; i++) {
            System.out.print("Nome " + (i+1) + ": ");
            nomes[i] = sc.nextLine().trim();
        }

        System.out.println("\n--- Resultados ---");
        for (String nome : nomes) {
            String formatado =
                nome.substring(0,1).toUpperCase() +
                nome.substring(1).toLowerCase();
            System.out.println("✓ " + formatado +
                " (" + nome.length() + " letras)");
        }
        sc.close();
    }
}
```



1. Coleta de Dados

Lê 4 nomes do usuário e aplica `trim()`



2. Formatação

Primeira letra em **MAIÚSCULA** + resto em **minúscula**



3. Resultado

Exibe nome formatado com contagem de letras



Exemplo de Saída

- ✓ Maria silva (11 letras)
- ✓ João santos (11 letras)
- ✓ ANA costa (9 letras)
- ✓ Pedro oliveira (14 letras)

Debugging com Arrays e Strings



ERRO 1

ArrayIndexOutOf BoundsException



Causa:

Tentar acessar índice inexistente

```
array[5] // array de 5 posições
```



ERRO 2

NullPointerException



Causa:

Usar array/String não inicializado



Sempre inicialize antes de usar



ERRO 3

Concatenação Acidental

```
"Resultado: " + 2 + 3  
→ "Resultado: 23" ✗
```

```
"Resultado: " + (2 + 3)  
→ "Resultado: 5" ✓
```



Dica do Debugger

Inspire o conteúdo do array a cada iteração – veja os valores nas posições!

Recapitulando – O que aprendemos hoje 📋



Arrays

- ✓ Declarar, criar e inicializar arrays
- ✓ Acessar elementos por índice (começa em 0!)
- ✓ Percorrer com **for** e **for-each**
- ✓ Propriedade **length**
- ✓ Arrays multidimensionais (matrizes)



Strings

- ✓ Métodos: **length()**, **toUpperCase()**, **toLowerCase()**, **trim()**
- ✓ Métodos: **contains()**, **indexOf()**, **substring()**, **replace()**, **split()**
- ✓ Comparação com **.equals()** e **.equalsIgnoreCase()**
- ✓ Conversão com **Integer.parseInt()**, **String.valueOf()**
- ✓ **StringBuilder** para construção eficiente

Sugestão de Estudo Extra 📖

Opcional – não é necessário para acompanhar a próxima aula



StringBuilder aprofundado

Métodos avançados de manipulação

`insert()`

`delete()`

`replace()`



Imutabilidade de String

Entender por que Strings não mudam em Java

`String Pool`



Classe Arrays

Utilitários prontos para arrays

`Arrays.sort()`

`Arrays.fill()`

`Arrays.toString()`



Formatação com `String.format()`

Gerar mensagens formatadas

```
String.format("Olá, %s! Você tem %d anos.", nome, idade);
```

Exercícios Práticos

Simulando ambiente real de desenvolvimento ágil

JAVA-001

Story

HIGH

Sistema de Lista de Tarefas

Implementar sistema de gerenciamento de tarefas com armazenamento em array e funcionalidades de CRUD

✔ Critérios de Aceitação:

- ☐ Array de Strings com capacidade para 10 tarefas
- ☐ Listar todas as tarefas numeradas
- ☐ Marcar tarefa como concluída

💡 Dica: Use arrays e métodos String

JAVA-002

Bug

HIGH

Validação de Email Falhando

O sistema está aceitando emails inválidos. Implementar validação robusta para emails do usuário

✔ Critérios de Aceitação:

- ☐ Verificar se contém "@" e "."
- ☐ "@" deve vir antes de "."
- ☐ Não começar nem terminar com "@"

💡 Dica: Use contains(), indexOf()

JAVA-003

Task

MEDIUM

Relatório de Vendas Mensal

Gerar relatório com estatísticas de vendas: total, média, melhor dia, pior dia

✔ Critérios de Aceitação:

- ☐ Array com vendas de 30 dias
- ☐ Calcular total e média de vendas
- ☐ Encontrar melhor e pior dia

💡 Dica: Arrays numéricos e estatísticas

JAVA-004

Epic

LOW

Busca de Produtos

Implementar sistema de busca com filtros e ordenação para catálogo de produtos

✔ Critérios de Aceitação:

- ☐ Buscar produto por nome (ignorar maiúsculas)
- ☐ Ordenar produtos por preço
- ☐ Listar produtos abaixo de R\$ 50

💡 Dica: Arrays 2D, equalsIgnoreCase()

🚀 Missão Cumprida!

Até a próxima aula! 📅



Próxima Aula

Métodos e Modularização

💡 Lembre-se

- ✓ **Arrays:** Índice começa em **0**
- ✓ **Strings:** Use **`.equals()`** para comparar
- ✓ Use **`.trim()`** antes de comparar entradas
- ✓ **Pratique!** Resolva pelo menos **3 exercícios** em casa 📅

” Inspiração

Programar é como resolver um **puzzle** – cada peça que você aprende abre **novas possibilidades!**



Continue praticando!